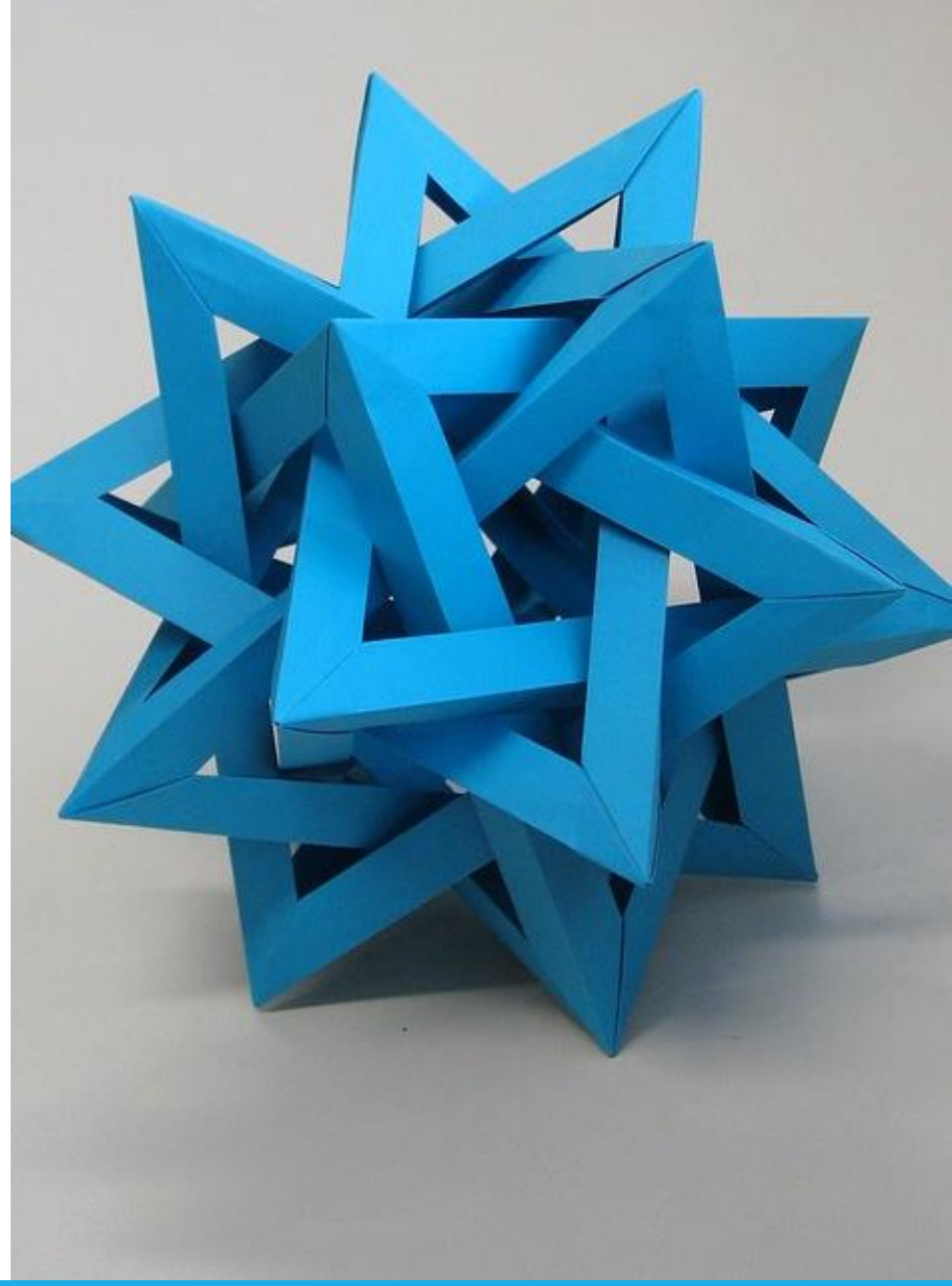
POLITECNICO
DI TORINO
Dipartimento
di Automatica e Informatica

Unit P8: Complex Data Structures

SETS, DICTIONARIES, AND COMBINATIONS OF DATA STRUCTURES

Chapter 8



Unit Goals

- To build and use a **set** container
- To learn common set **operations** for processing data
- To build and use a **dictionary** container
- To work with a dictionary for table **lookups**

In this unit, we will learn how to work with two more types of containers (**sets** and **dictionaries**) as well as how to combine containers to model complex structures.

Contents

- Sets
- Dictionaries
- Complex Structures

Sets

8.1

Sets

- A set is a container that stores a **collection of unique values**
  - **Unique → No duplicates!!**
- Unlike a list, the elements or members of the set are not stored in any particular order and cannot be accessed by position
- Operations are the same as the operations performed on sets in mathematics
- Because sets do not need to maintain a particular order, set operations are much faster than the equivalent list operations

Example Set

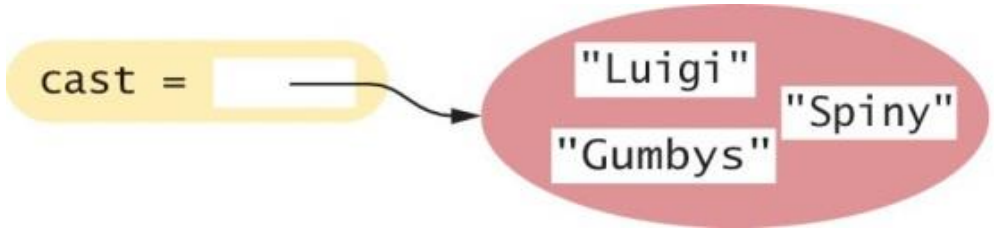
- Each of these sets contains some colors—the colors of the British, Canadian, and Italian flags
- In each set, the order does not matter, and the colors are not duplicated in any one of the sets

```
uk_flag = { 'blue', 'red', 'white' }
ca_flag = { 'red', 'white' }
it_flag = { 'green', 'red', 'white' }
```

Creating and Using Sets

- To create a set with initial elements, you can specify the elements enclosed in braces, just like in mathematics:

```
cast = { "Luigi", "Gumbys", "Spiny" }
```



- Alternatively, you can use the set() function to convert any sequence (a list, tuple, string, etc., etc.) into a set:

```
names = ["Luigi", "Gumbys", "Spiny"]
cast = set(names)
```

Creating and using Sets

- Examples of the set() function:

```
nums = [1,2,3,4,2,1,2]
s= set(nums)   # s={1,2,3,4}
```

```
Name = “Anna”
s = set(Name) # s = {‘A’,’n’,’a’}
```

Allowed Values in Sets

- For technical reasons related to the implementation of sets in Python, the **values inside a set (and the keys of a dictionary, see later) must be:**
  - **Immutable**
  - **Comparable**

- **OK**: numbers (int or float), strings, tuples, ….
- **NOT OK**: lists, dictionaries, etc…

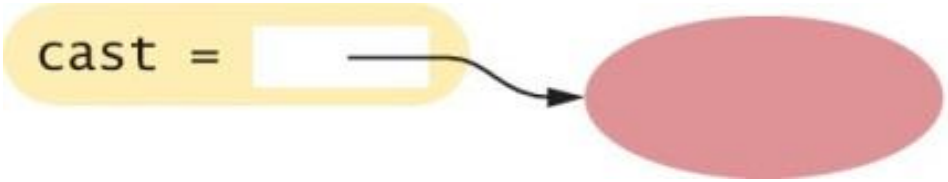
- If you're curious, look at: https://docs.python.org/3/glossary.html#term-hashable

Creating an Empty Set

- For historical reasons, you cannot use `{}` to make an empty set in Python (it refers to a **dictionary**)
- Instead, use the `set()` function with no arguments:

```
cast = set()
```

- As with any container, you can use the `len()` function to obtain the number of elements in a set:




```
numberOfMovieCharacters = len(cast)   # In this case it's zero
```

Set Membership: `in`

- To determine whether an element is contained in the set, use the `in` operator or its inverse, the `not in` operator:

```
if "Luigi" in cast :
    print("Luigi is a character in Monty Python’s Flying Circus.")
else :
    print("Luigi is not a character in the show.")
```

Accessing Set Elements

- Because sets are unordered, you cannot access the elements of a set by position as you can with a list
- We use a `for` loop to iterate over the individual elements:

```
print("The cast of characters includes:")
for character in cast :
    print(character)
```

- Note that the order in which the elements of the set are visited depends on how they are stored internally – it's unpredicable

Accessing Set Elements (2)

- For example, the previous loop above displays the following:

```
The cast of characters includes:
Gumbys
Spiny
Luigi
```

- Note that the **order** of the elements in the output is different from the order in which the set was created

Displaying Sets In Sorted Order

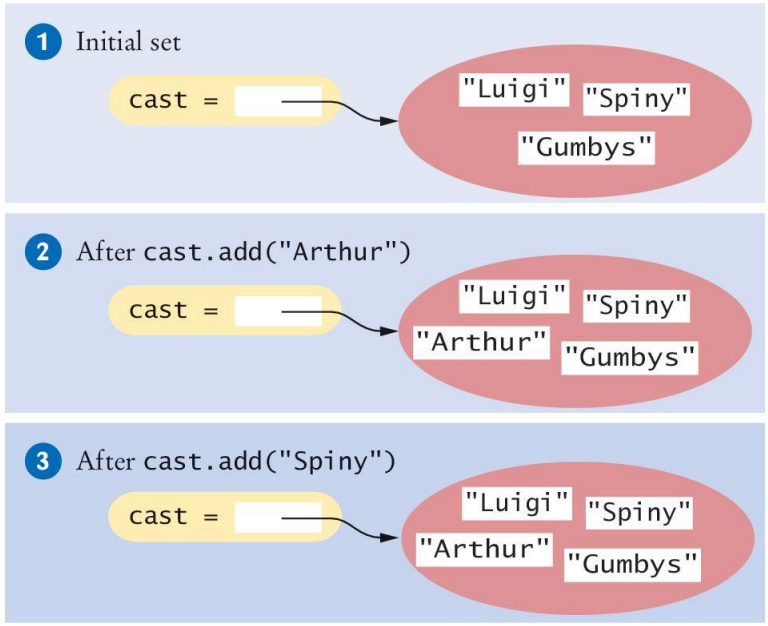
- Use the `sorted()` function, which returns a list (not a set) of the elements in sorted order
  - Numbers: increasing order (by default, or decreasing with reverse=True)
  - Strings: lexicographical order
- The following loop prints the cast in sorted order:

```
for actor in sorted(cast) :
    print(actor)
```

Adding Elements

- Sets are mutable collections, so you can add elements by using the `add()` method:

```
cast = set(["Luigi", "Gumbys", "Spiny"])   #1
cast.add("Arthur")                         #2
cast.add("Spiny")                          #3
```



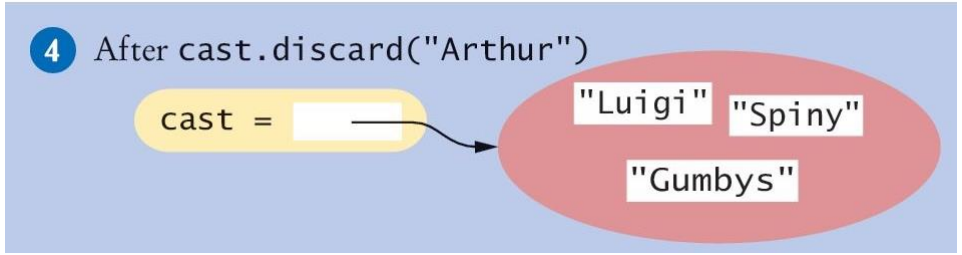
Arthur is not in the set, so it is added to the set and the size of the set is increased by one

Spiny is already in the set, so there is no effect on the set (**no duplicates allowed)**

Removing Elements: `discard()`

- The `discard()` method removes an element if the element exists:

```
cast.discard("Arthur")  #4
```



- It has no effect if the given element is not a member of the set:

```
cast.discard("The Colonel")   # Has no effect
```

Removing Elements: `remove()`

- The `remove()` method, on the other hand
  - removes an element if it exists
  - but raises an exception if the given element is not a member of the set:

```
cast.remove("The Colonel")   # Raises an exception
```

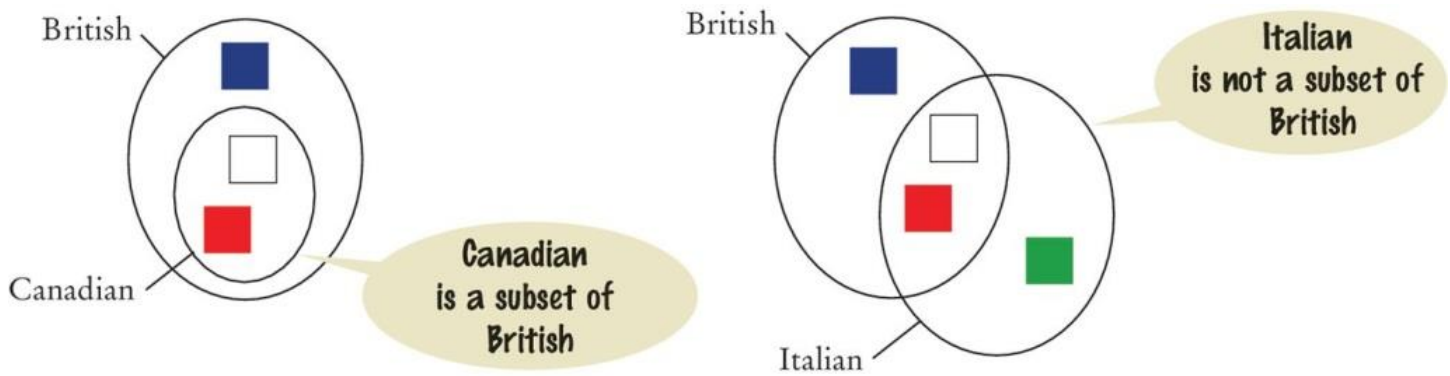
Removing Elements: `clear()`

- The `clear()` method removes all elements of a set, leaving the empty set:

```
cast.clear() # cast now has size 0
```

Subsets

- A set is a subset of another set if and only if every element of the first set is also an element of the second set
- In the image below, the Canadian flag colors are a subset of the British colors
- The Italian flag colors are not



The `issubset()` Method

- The `issubset()` method returns `True` or `False` to report whether one set is a subset of another:

```
canadian = { "Red", "White" }
british = { "Red", "Blue", "White" }
italian = { "Red", "White", "Green" }

# True
if canadian.issubset(british) :
    print("All Canadian flag colors appear in the British flag.")

# True
if not italian.issubset(british) :
    print("At least one of the colors in the Italian flag does
       not.")
```

Set Equality / Inequality

- We test set equality with the "==" and "!=" operators
- Two sets are equal if and only if they have exactly the same elements

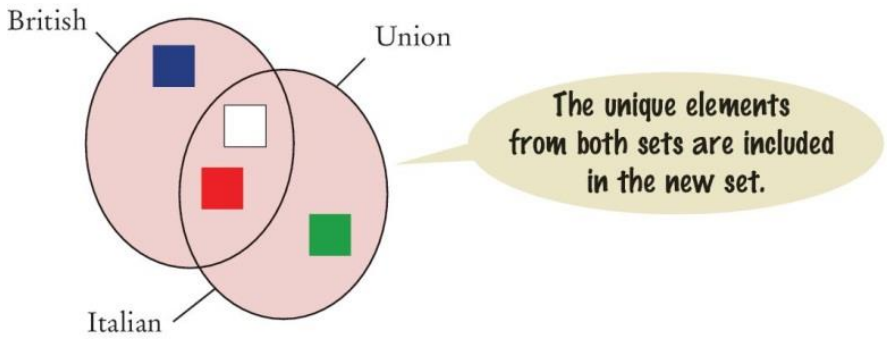
```
french = { "Red", "White", "Blue" }
if british == french :
    print("The British and French flags use the same colors.")
```

Set Union: `union()`

- The union of two sets contains all of the elements from both sets, with duplicates removed

```
# inEither: The set {"Blue", "Green", "White", "Red"}
inEither = british.union(italian)
```

- Both British and Italian sets contain Red and White, but the union is a set and therefore contains only one instance of each color

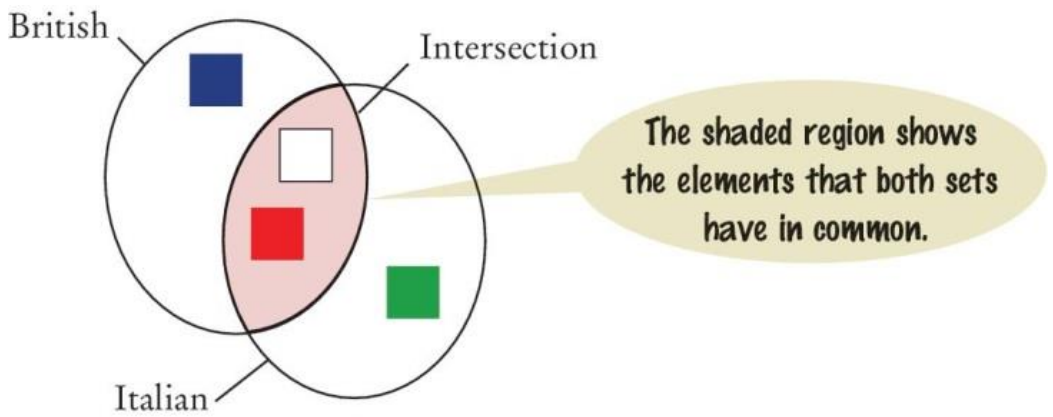


Note that the union() method returns a new set. It does not modify either of the sets in the call

Set Intersection: `intersection()`

- The intersection of two sets contains all of the elements that are in both sets

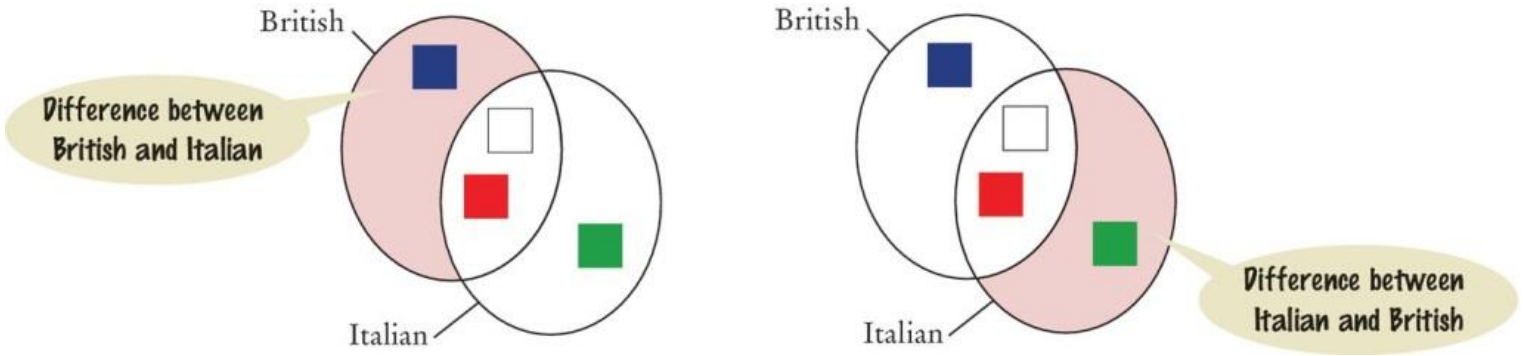
```
# inBoth: The set {"White", "Red"}
inBoth = british.intersection(italian)
```



Difference of Two Sets: `difference()`

- The difference of two sets results in a new set that contains those elements in the first set that are not in the second set

```
print("Colors that are in the Italian flag but not the
   British:")
print(italian.difference(british)) # Prints {'Green'}
```

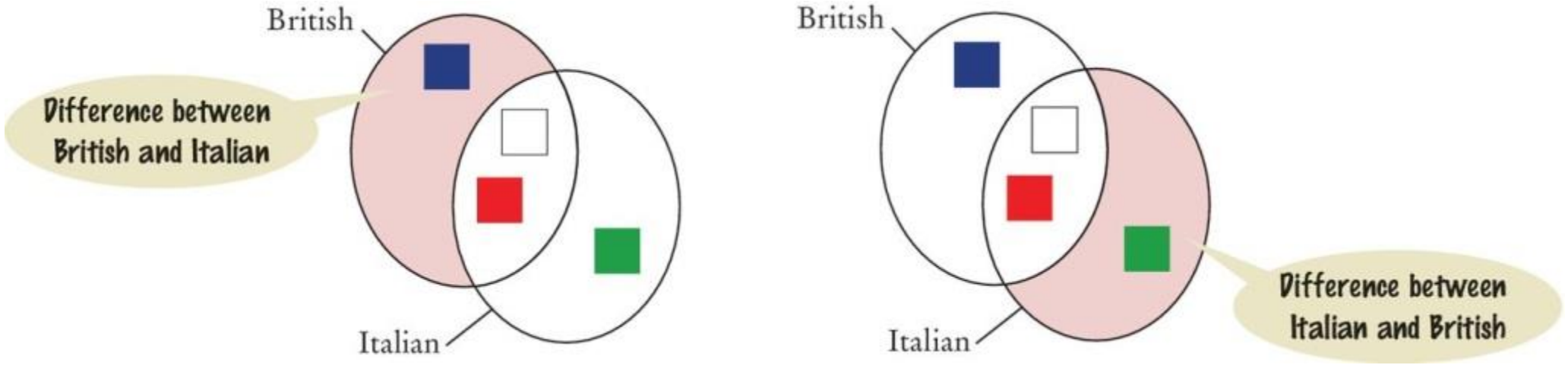


Difference of Two Sets: `difference()`

- Careful: `difference` is **not commutative**

```
s1 = { "Red", "Blue", "White" }
s2 = { "Red", "White", "Green" }

print(s1.difference(s2)) #{'Blue'}
print(s2.difference(s1)) #{'Green'}
```



Common Set Operations

Table 1 Common Set Operations

Operation	Description

`s = set()`
`s = set(seq)`
`s = {e1, e2, ..., en}`
	Creates a new set that is either empty, a duplicate copy of sequence *seq*, or that contains the initial elements provided.

`len(s)`	Returns the number of elements in set *s*.
element `in s` *element* `not in s`	Determines if *element* is in the set.
`s.add(`*element*`)`	Adds a new element to the set. If the element is already in the set, no action is taken.
`s.discard(`*element*`)` `s.remove(`*element*`)`	Removes an element from the set. If the element is not a member of the set, `discard` has no effect, but `remove` will raise an exception.
`s.clear()`	Removes all elements from a set.
`s.issubset(t)`	Returns a Boolean indicating whether set *s* is a subset of set *t*.

Common Set Operations (2)

Table 1 Common Set Operations	
`s == t` `s != t`	Returns a Boolean indicating whether set *s* is equal to set *t*.
`s.union(t)`	Returns a new set that contains all elements in set *s* and set *t*.
`s.intersection(t)`	Returns a new set that contains elements that are in *both* set *s* and set *t*.
`s.difference(t)`	Returns a new set that contains elements in *s* that are not in set *t*.

Remember: `union`, `intersection` and `difference` return new sets
They do not modify the set they are applied to

Set operations using short operators

Set operation using methods	Operators equivalence
`x1.union(x2)`	`x1 | x2`
`x1.intersection(x2)`	`x1 & x2`
`x1.difference(x2)`	`x1 - x2`
`x1.symmetric_difference(x2)`	`x1 ^ x2`
`x1.issubset(x2)`	`x1 <= x2`
	`x1 < x2`
`x1.issuperset(x2)`	`x1 >= x2`
	`x1 > x2`

Set Operations

Function	Description
all()	Returns `True` if all elements of the set are true (or if the set is empty).
any()	Returns `True` if any element of the set is true. If the set is empty, returns `False`.
enumerate()	Returns an enumerate object. It contains the index and value for all the items of the set as a pair.
len()	Returns the length (the number of items) in the set.
max()	Returns the largest item in the set.
min()	Returns the smallest item in the set.
sorted()	Returns a new sorted list from elements in the set(does not sort the set itself).
sum()	Returns the sum of all elements in the set.

Method	Description
add()	Adds an element to the set
clear()	Removes all elements from the set
copy()	Returns a copy of the set
difference()	Returns the difference of two or more sets as a new set
difference_update()	Removes all elements of another set from this set
discard()	Removes an element from the set if it is a member. (Do nothing if the element is not in set)
intersection()	Returns the intersection of two sets as a new set
intersection_update()	Updates the set with the intersection of itself and another
isdisjoint()	Returns `True` if two sets have a null intersection
issubset()	Returns `True` if another set contains this set
issuperset()	Returns `True` if this set contains another set
pop()	Removes and returns an arbitrary set element. Raises `KeyError` if the set is empty
remove()	Removes an element from the set. If the element is not a member, raises a `KeyError`
symmetric_difference()	Returns the symmetric difference of two sets as a new set
symmetric_difference_update()	Updates a set with the symmetric difference of itself and another
union()	Returns the union of sets in a new set
update()	Updates the set with the union of itself and others

https://www.programiz.com/python-programming/dictionary

💡 Programming Tip

- In programs that need to handle collections of **unique** elements, sets are much more efficient than lists!
- Some programmers are used to lists, and instead of:

```
itemSet.add(item)
```

- they write:

```
if (item not in itemList)
    itemList.append(item)
```

- In this way, the resulting program is much slower (more than 10 times!!!)

Counting Unique Words

Problem Statement

- We want to be able to count the number of unique words in a text document
  - The song “Mary had a little lamb” has 57 unique words
- Our task is to write a program that reads in a text document and determines the number of unique words in the document

Step One: Understand the Task

- To count the number of unique words in a text document we need to be able to determine if a word has been encountered earlier in the document
  - Only *the first occurrence* of a word should be *counted*
- The easiest way to do this is to read each word from the file and add it to a set
  - Because a set **cannot contain duplicates** we can use the add method
  - The add method will prevent a word that was encountered earlier from being added to the set
- After we process every word in the document the size of the set will be the number of unique words contained in the document

Step Two: Decompose the Problem

- Create an empty set
- for each word in the text document
  - Add the word to the set
- Number of unique words = the size of the set
- Creating the empty set, adding an element to the set, and determining the size of the set are standard set operations
- Reading the words in the file can be handled as a separate function

Step Three: Build the Set

- We need to read individual words from the file. For simplicity in our example we will use a literal file name

```
inputFile = open("nurseryrhyme.txt", "r")
For line in inputFile :
    theWords = line.split()
    For words in theWords :
        Process word
```

- To count unique words we need to remove any nonletters and remove capitalization
- We will design a function to "clean" the words before we add them to the set

Step Four: Clean the Words

- To strip out all the characters that are not letters we will iterate through the string, one character at a time, and build a new “clean” word

```
def clean(string) :
    result = ''
    for char in string :
        if char.isalpha() :
            result = result + char
return result.lower()
```

Step Five: Put Everything Together

- Implement the main() function and combine it with the other functions

Spell checking

Set Example: Spell Checking

- Check the correct “spelling” of a document, printing all the “wrong” words.
- Data:
  - `words.txt`: text file containing a dictionary of correct words (one per line)
  - `story.txt`: file containing the text to be checked.
- **Output:** all words in the story that are not present in the dictionary.

Solution Design

1. **Read each word in the dictionary to create a set of strings.**
2. **Read the story and:**
   - Isolate single words, removing spaces and punctuation
   - Store the words in **a second set**
3. Compute the difference between the two sets: words that are in the story, but not in the dictionary.
4. Print the difference.

Notes: handle uppercase/lowercase, and remember that set difference is asymmetric!

Example: `Spellcheck.py`

```
1  ##
2  #  This program checks which words in a file are not present in a list of
3  #  correctly spelled words.
4  #
5
6  #  Import the split function from the regular expression module.
7  from re import split
8
9  def main() :
10    # Read the word list and the document.
11    correctlySpelledWords = readWords("words")
12    documentWords = readWords("alice30.txt")
13
14    # Print all words that are in the document but not the word list.
15    misspellings = documentWords.difference(correctlySpelledWords)
16    for word in sorted(misspellings) :
17       print(word)
```

Example: `Spellcheck.py`

```
24  def readWords(filename) :
25     wordSet = set()
26     inputFile = open(filename, "r")
27
28     for line in inputFile :
29        line = line.strip()
30        # Use any character other than a-z or A-Z as word delimiters.
31        parts = split("[^a-zA-Z]+", line)
32        for word in parts :
33           if len(word) > 0 :
34              wordSet.add(word.lower())
35
36     inputFile.close()
37     return wordSet
38
39  # Start the program.
40  main()
```

Execution: `Spellcheck.py`

```
...
champaign
chatte
clamour
comfits
conger
croqueted
croqueting
cso
daresay
dinn
dir
draggled
dutchess
...
```

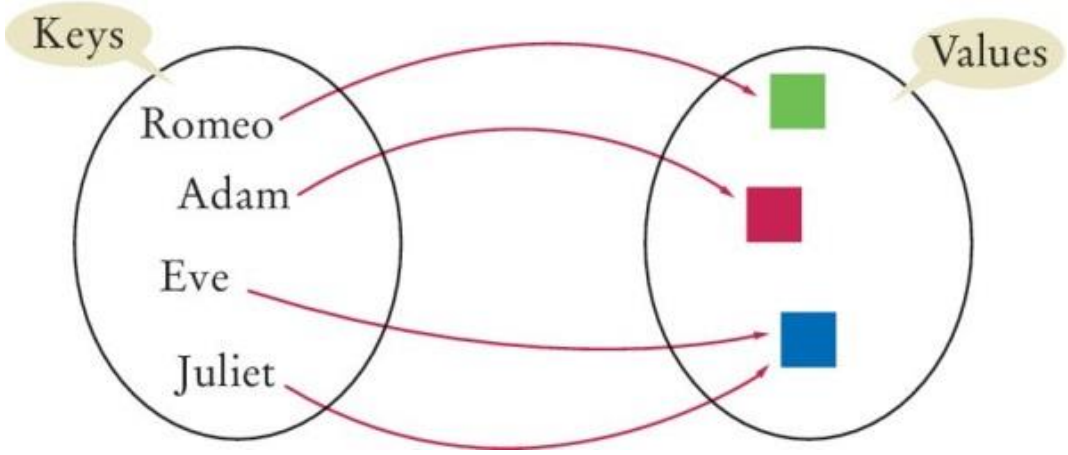
Dictionaries

8.2

SECTION 8.2

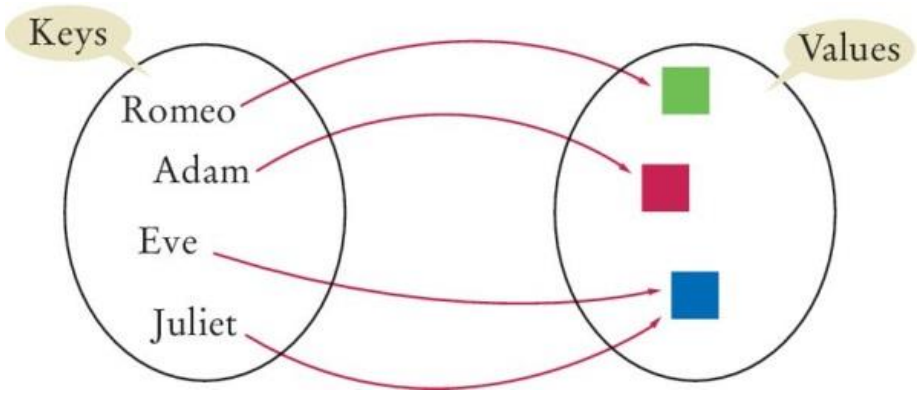
Dictionaries

- A dictionary is a container that keeps associations between keys and values
  - Also called *associative array* or *mapping*
- Every key in the dictionary has an associated value
- **Keys are unique**, but the same value may be associated with several keys
- Example (the mapping between key and value is indicated by an arrow):



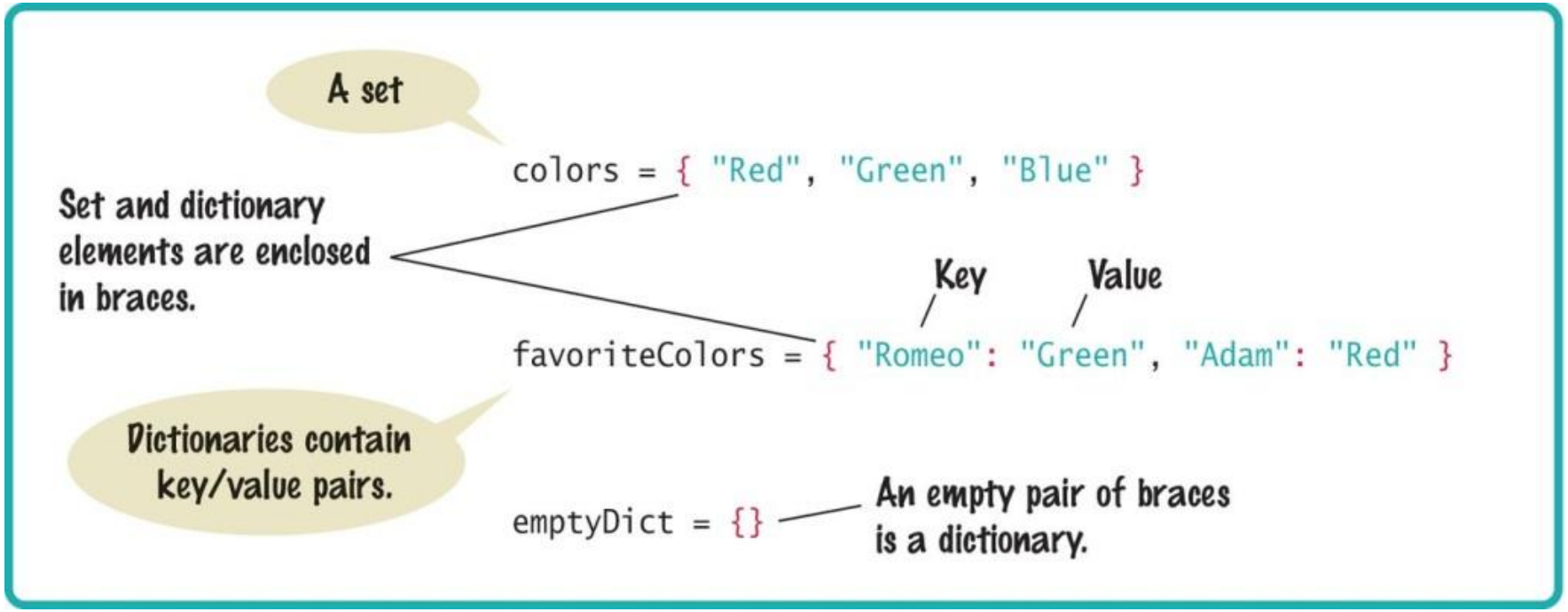
Dictionary syntax

- List the associations between braces (like for sets)
  - Each association is formatted as ***key : value***
  - The presence of these associations (and of the :) distinguishes a dictionary from a set from the point of view of Python syntax



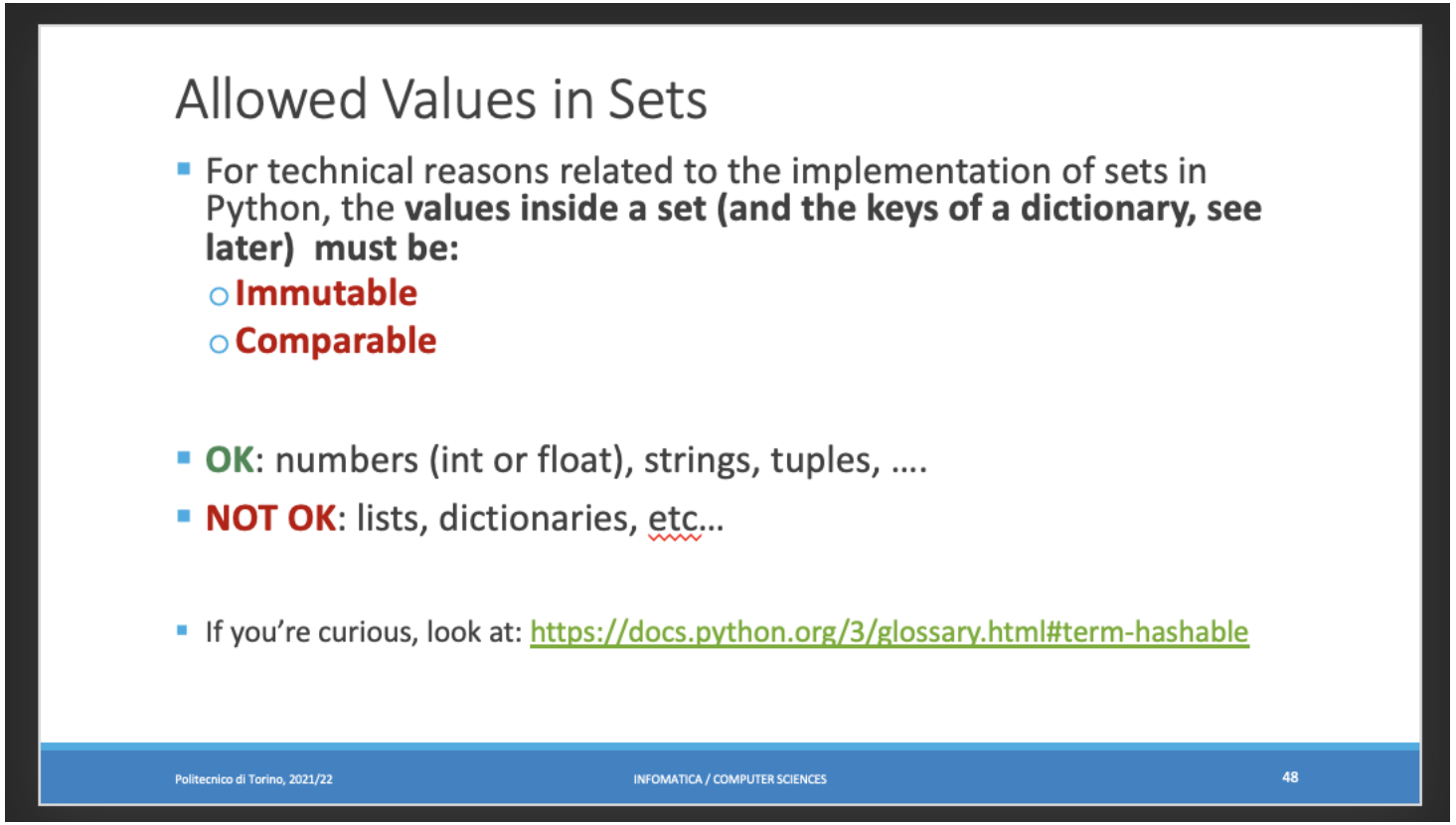

```
{
  "Romeo": "green",
  "Adam": "purple",
  "Eve": "blue",
  "Juliet": "blue"
}
```

Syntax: Sets and Dictionaries



Allowed Keys and Values in Dictionaries

- Remember…



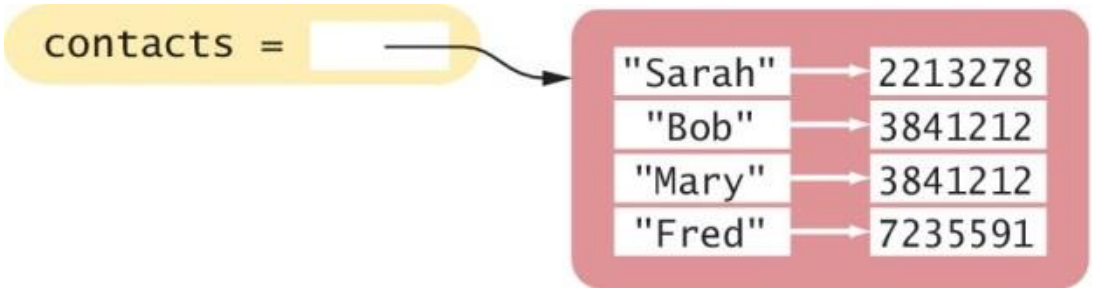
- Dictionary keys must be immutable. Usually:
  - Strings (names, alphanumeric codes, …)
  - Numeri (IDs, …)

- Dictionary **values** can be anything
  - e.g. numers, strings, lists, other dictionaries, …

Example

- Suppose you need to write a program that looks up the phone number for a person in your contact list
- You can use a dictionary where the names are keys and the phone numbers are values

```
contacts = { "Fred": 7235591, "Mary": 3841212, "Bob":
   3841212, "Sarah": 2213278 }
```



Duplicating Dictionaries: `dict()`

- You can create a duplicate copy of a dictionary using the `dict()` function:

```
oldContacts = dict(contacts)
```

- You can create an empty dictionary with:

```
contacts = dict()
contacts = {}
```

Accessing Dictionary Values [ ]

- The subscript operator `[ ]` is used to return the value associated with a key
  - Syntax: `dictionary [ key ]`

NOTE THAT WE USE THE KEY HERE, NOT THE POSITION AS IN A LIST OR TUPLE

```
# prints 7235591.
print("Fred's number is", contacts["Fred"])
```

- The key may be:
  - A literal value
  - The content of a variable or the result of an expression

```
s = “Mary”
print(“Mary’s number is”, contacts[s])
```

Accessing Dictionary Values [ ]

- Note that the dictionary is not a sequence-type container like a list.
  - **You cannot access the items by index or position**
  - A value can only be accessed using its associated key

*If you try to access the dictionary using a key that does not exist, a **KeyError** exception will be raised*

Dictionaries: Checking Membership

- To find out whether a key is present in the dictionary, use the `in` (or `not in`) operator:

```
if "John" in contacts:
    print("John's number is", contacts["John"])
else :
    print("John is not in my contact list.")
```

- Prevent exceptions in case the key does not exist…

Default Values

- As a shortcut, you may use the `get` method to access the dictionary, which allows you to also specify a default value
  - Syntax: `dictionary.get(key, default)`
  - The default value is returned if the key is not found

```
number = contacts.get("Tony", "missing")
```

  - Equivalent to:

```
if "Tony" in contacts:
  number = contacts["Tony"]
else:
  number = "missing"
```

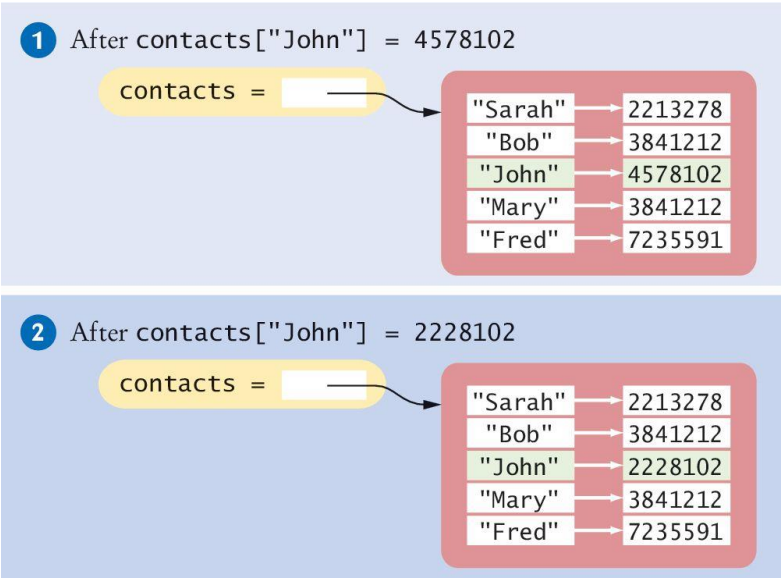
Adding/Modifying Items

- A dictionary is a mutable container
- You can add a new item using the subscript operator [ ]
  - **Note that this can't be done with a list!**

```
contacts["John"] = 4578102   #1
```

- To change the value associated with a given key, set a new value using the [ ] operator on an existing key:

```
contacts["John"] = 2228102   #2
```



Adding New Elements Dynamically

- Sometimes you may not know which items will be contained in the dictionary when it's created
  - For example: data will be acquired from the user or a file…
- You can create an empty dictionary :

```
favoriteColors = {}
```

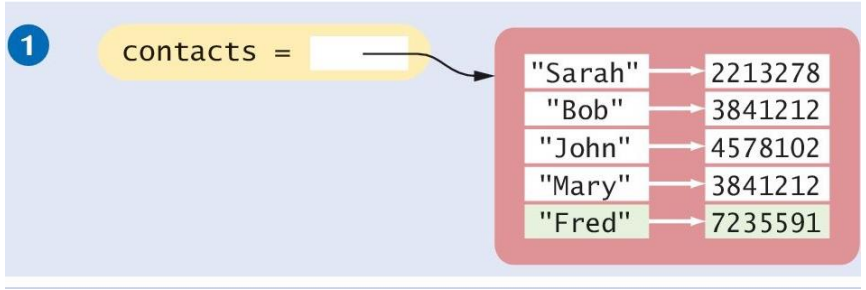
- and add new items later, as needed:

```
favoriteColors["Juliet"] = "Blue"
favoriteColors["Adam"] = "Red"
favoriteColors["Eve"] = "Blue"
favoriteColors["Romeo"] = "Green"
```

Removing Elements

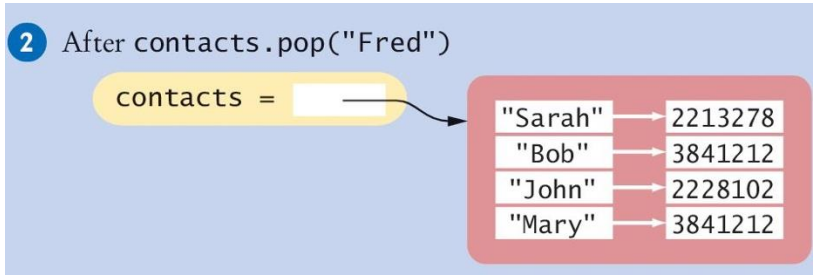
- To remove an item from a dictionary, call the pop( ) method with the key as the argument:

```
contacts = { "Fred":
   7235591, "Mary": 3841212,
   "Bob": 3841212, "Sarah":
   2213278 }
```



- This removes the entire item, both the key and its associated value

```
contacts.pop("Fred")
```



Removing Elements

- The `pop()` method returns the value of the item being removed, so you can use it or store it in a variable:

```
fredsNumber = contacts.pop("Fred")
```

- Note: If the key is not in the dictionary, the `pop` method raises a `KeyError` exception
  - To prevent the exception from being raised, you should test for the key in the dictionary:

```
if "Fred" in contacts :
    contacts.pop("Fred")
```

Traversing a Dictionary

- You can iterate over the **individual keys** in a dictionary using a `for` loop:

```
print("My Contacts:")
for key in contacts :
    print(key)
```

- The result of this code fragment is :

```
My Contacts:
Sarah
Bob
John
Mary
Fred
```

Note that the dictionary stores its items in an order that is optimized for efficiency, which may not be the order in which they were added

Traversing a Dictionary

```
print(“My contacts:”)
for key in contacts :
  print(key, contacts[key])
```

```
My contacts:
Sarah 2213278
Bob 3841212
John 4578102
Mary 3841212
Fred 7235591
```

Traversing a Dictionary: In Order

- To iterate through the keys in sorted order, you can use the `sorted()` function as part of the `for` loop :
- Now, the contact list will be printed in order by name:

```
My Contacts:
Bob 3841212
Fred 7235591
John 4578102
Mary 3841212
Sarah 2213278
```

```
print("My Contacts:")
for key in sorted(contacts) :
    print(key, contacts[key])
```

Returns a list!!

Traversing a Dictionary “By Value”

- If needed, you can also iterate over the values using the `values()` method:

- Example:

```
for number in contacts.values() :
        #use number (e.g. print, add to a list, etc)
```

- Can be useful in some cases:
  - Example: compute maximum value stored in a dictionary:
  - `max(contacts.values())`

Getting both keys and values

- Python allows you to iterate over the items in a dictionary using the `items()` method
- This is a bit more efficient than iterating over the keys and then looking up the value of each key
- The `items()` method returns a sequence of tuples that contain the keys and values of all items
  - Here the loop variable item will be assigned a tuple that contains the key in the first slot and the value in the second slot

```
for item in contacts.items():
    print(item[0], item[1])
```

```
for key, val in contacts.items():
    print(key, val)
```

Getting both keys and values

- Python allows you to iterate over the items in a dictionary using the `items()` method
- Useful and efficient, but not strictly necessary. You can always scan by key and get the value with []
- Here the loop variable item will be assigned a tuple that contains the key in the first slot and the value in the second slot

```
for item in contacts.items():
    print(item[0], item[1])
```

```
for key, val in contacts.items():
    print(key, val)
```

Common Dictionary Operations (1)

Table 2 Common Dictionary Operations

Operation	Returns
`d = dict()` `d = dict(c)`	Creates a new empty dictionary or a duplicate copy of dictionary *c*.
`d = {}` `d = {k1: v1, k2: v2, ..., kn: vn}`	Creates a new empty dictionary or a dictionary that contains the initial items provided. Each item consists of a key (*k*) and a value (*v*) separated by a colon.
`len(d)`	Returns the number of items in dictionary *d*.
key `in` *d* *key* `not in` *d*	Determines if the key is in the dictionary.
d[*key*] = *value*	Adds a new *key*/*value* item to the dictionary if the *key* does not exist. If the key does exist, it modifies the value associated with the key.
`x = d[key]`	Returns the value associated with the given key. The key must exist or an exception is raised.

Common Dictionary Operations (2)

Table 2 Common Dictionary Operations

d.get(*key*, *default*)	Returns the value associated with the given key, or the default value if the key is not present.
d.pop(*key*)	Removes the key and its associated value from the dictionary that contains the given key or raises an exception if the key is not present.
d.values()	Returns a sequence containing all values of the dictionary.

Dictionary Operations

Function	Description
all()	Return `True` if all keys of the dictionary are True (or if the dictionary is empty).
any()	Return `True` if any key of the dictionary is true. If the dictionary is empty, return `False`.
len()	Return the length (the number of items) in the dictionary.
cmp()	Compares items of two dictionaries. (Not available in Python 3)
sorted()	Return a new sorted list of keys in the dictionary.

Method	Description
clear()	Removes all items from the dictionary.
copy()	Returns a shallow copy of the dictionary.
fromkeys(seq[, v])	Returns a new dictionary with keys from `seq` and value equal to `v` (defaults to `None`).
get(key[,d])	Returns the value of the `key`. If the `key` does not exist, returns `d` (defaults to `None`).
items()	Return a new object of the dictionary's items in (key, value) format.
keys()	Returns a new object of the dictionary's keys.
pop(key[,d])	Removes the item with the `key` and returns its value or `d` if `key` is not found. If `d` is not provided and the `key` is not found, it raises `KeyError`.
popitem()	Removes and returns an arbitrary item (**key, value**). Raises `KeyError` if the dictionary is empty.
setdefault(key[,d])	Returns the corresponding value if the `key` is in the dictionary. If not, inserts the `key` with a value of `d` and returns `d` (defaults to `None`).
update([other])	Updates the dictionary with the key/value pairs from `other`, overwriting existing keys.
values()	Returns a new object of the dictionary's values

https://www.programiz.com/python-programming/dictionary

Dictionaries as "associative arrays" 

Associative Array

	List (Array)	Dictionary (Associative Array)
Data structure type	Sequence	Mapping
Stored information	Sequence of values	Set of pairs (key, value)
Keys/Indexes	Integer index between `0` and `len()-1`	Immutable key: number, string, tuple
Values	Any	Any

- A dictionary is a good candidate for storing information where each element is identifiable by a unique key value.

- We can imagine it as a list, where the index is not necessarily numeric (it can be a string, a tuple , ...)

Example (1)

- Let's calculate the frequency with which names appear in a list of people

- ***Key*** : person name (string)
- ***Value*** : number of times it appears (integer)

```
count = {
'Jimmy': 3,
'Timmy': 2,
'Tommy': 4
}


count[name] = count[name]+1
```

Example (2)

- Each student has an ID number. This ID number is associated with their real name

- ***Key*** : ID number (integer)
- ***Value*** : student name (string)

- **Note: The keys are integers, but not consecutive, nor between 0 and len()-1**

```
students = {
123456: 'Mario Rossi',
211003: 'Paolo Bianchi',
234321: 'Anna Verdi'
}
```

Example (2a)

- Each student has an ID number. This ID number is associated with their real name **and surname**

- ***Key*** : serial number (integer)
- ***Value*** : student's last name and first name (list, with 2 string elements)

```
students = {
123456: ['Mario', 'Rossi'],
211003: ['Paul', 'Bianchi'],
234321: ['Anna', 'Verdi']
}
```

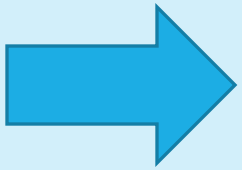
Example (3)

- In a game of chess, different pieces are placed on different squares
- ***Key*** : coordinates of the square (tuple containing two integers: row and column)
- ***Value*** : piece type (string)

```
pieces = {
(1,3): 'King',
(2,2): 'Pedestrian',
(8,8): 'Tower'
}
```

Exercise

- Compute the frequency of occurrence of each name in a file called names.txt (histogram)
  - Mario
  - Giulia
  - Mario
  - Matteo
  - Mario
  - Matteo



Mario: 3
Giulia: 1
Matteo: 2

Compare a solution using lists and one using dictionaries!

Dictionaries as “data records”

Storing Data Records

- **Data records**, in which each **record** consists of multiple **fields**, are very common
- We already know how to store this type of data in a table (list of lists)

```
person = [ 'John', 'Doe', 1971, 'New York', 'Student' ]
```

- But this requires remembering in **which element of the list each field is stored**
  - This can introduce run-time errors if you use the wrong list element when processing the record
  - `person[1]` is the last name… or is it `person[2]`?
- **Using a dictionary to store a data record solves this problem!!**

Dictionaries: Data Records

- You create a dictionary for each record:
  - **Key = field name**
  - **Value = data associated to that field**
- For example, this dictionary named record stores a single student record :

```
person1 = {
' firstName ': 'John',
' lastName ': 'Doe',
' birthdate ': 1971,
' city ': 'New York',
' profession ': 'Student' }
```

Remember that the key must be a string, don't forget the quotes

- You may then create a list of such records (**list of dictionaries**)

```
people = [ person1, person2, person3, ... ]
```

Reading from files containing data records

- To extract data records from a file, we can define a function that reads a single record and returns it as a dictionary
- E.g.: if the file contains records madeup of country names and population separated by colon

```
Afghanistan:32738376
Akrotiri:15700
Albania:3619778
Algeria:33769669
American Samoa:57496
Andorra:72413
Angola:12531357
Eel:14108
```

```
def extract_record ( infile ):
   record = {}
   line = infile.readline ()
       if line != "" :
   fields = line.split ( ":" )
   record[ "country" ] = fields[ 0 ]
   record[ "population" ] = int (fields[ 1 ])
       return record
```

Reading from files containing data records (2)

- The dictionary created by the `extract_record` function has two elements, one with key `"country"` and the other with key `"population"`
- The result of this function can be used to print the records to the terminal.

```
infile = open("populations.txt", "r")
record = extract_record(infile)
while len(record) > 0 :
    print(f"{record['country']:20} {record[ 'population']:10 }")
    record = extract_record(infile)
```

- Or to store them in a list (`list.append (record)`)

Example

- Store this CSV file as a **list of dictionaries**

ID	Surname	Name	Score
262062	McGill	Jimmy	30
274724	Wexler	Kim	23
285318	Pinkman	Jesse	27
293372	White	Skyler	18
295644	Fring	Gus	24
295880	Schrader	Marie	30
298624	Ehrmantraut	Mike	29

- ..where the keys are taken from the **column names.**
  - For example, the first record will be:
    - `{'ID': '262062', 'Surname': 'McGill', 'Name': 'Jimmy', 'Score': 30}`

List of Dictionaries

```
for line in f:
    record = {}   #empty dictionary
    line = line.strip()
    fields = line.split(",“) # list of strings
    for i in range(len(attributes)):
          record[attributes[i]] = fields[i]
    list.append(record)
```

attributes is a list containing the column names (can be extracted from the first line of the file)

List of Lists (standard ‘table’)

```
f = open('test.txt','r')
list = []
for line in f:
    if line != "" :
        line = line.strip()
        fields = line.split(",")
  list.append(fields)
```

Sorting a list of dictionaries?

- The list is filled in the same order in which the elements appear in the file.
- How to sort the list of students by name? By ID? By surname?
- This does not work: `students.sort()`

  `TypeError : '<' not supported between instances of ' dict ' and ' dict ‘`
  - Two `dict` data structures cannot be compared with each other , so the sort method does not know how to behave.

Advanced topic

Advanced sorting

HOW TO ORDER LISTS OF DICTIONARIES AND LISTS OF LISTS OR TUPLES USING THE PARAMETER *KEY=*

Sorting HOW TO
https://docs.python.org/3/howto/sorting.html

https://realpython.com/python-sort/
https://realpython.com/sort-python-dictionary/

Premise

- A list can to be ordered :
  - With the `list.sort ()` method: in place
  - With the `sorted(list)` function: returns a new list , sorted
- In both cases, the internal sorting algorithms apply multiple time a comparison between two elements of the list, of the form:
  - `if list [i] < list[j]`
- So, the result of sorting depends from the behavior of the comparison operator ' `<` '
  - This behavior depends , in turn , on the type of data contained in the list
  - Examples (already seen): numbers → numerical sort, strings → lexicographical sort, etc…

Premise

- A list can to be ordered :
  - With the `list.sort ()` method: in place
  - With the `sorted(list)` function: returns a new list , sorted
- In both cases, the internal sorting
  comparison between two element
  - `if list [i] < list[j]`
- So, the result of sorting depends fi
  comparison operator ' `<` '
  - This behavior depends , in turn , on the type of data contained in the list
  - Examples (already seen): numbers → numerical sort, strings → lexicographical sort, etc…

What happens if my list contains complex data?

Examples:
A list of lists (table)?
A list of dictionaries ?

Sorting criteria

Data type content in the list	Ordering resulting
`int , float`	**Numerical** ascending
`complex`	Impossible – complex numbers are not sortable `TypeError : '<' not supported between instances of 'complex' and 'complex'`
`str`	**Lexicographical** ascending , based on Unicode codes
`list , tuple`	Uses the comparison criteria between lists and tuples. See Unit P6.
`dict`	Impossible – Dictionaries are not comparable `TypeError : '<' not supported between instances of ' dict ' and ' dict '`
`set`	Unpredictable – does not generate errors , but the result is useless, unless the different sets are subsets of one another (in fact `<` is synonymous with `issubset`)

To reverse the sorting, we add the **`reverse=True`** optional parameter.

Sort keys (I)

- It is possible to specify to the sort method (and sorted function) **which sorting criteria they must use**, to customize the sorting
  - Useful for sorting lists of dictionaries
  - Useful for changing the 'default' sorting behaviour for lists of lists

- In these cases you need to provide a " sort key ", using the `key parameter`
  - `students.sort ( key= ... )`

Sorting keys (II)

```
list .sort (key= ... )
```

- The key , *in general* , is a function , computable for each single record (dictionary), whose value depends only on the data stored in it.

- If we specify a key, the sorting algorithm will apply the function before comparing the elements:
  - Without key: `list[i] < list[j]`
  - With key: `key(list[i]) < key(list[j])`

- The sorting will then take place by ascending values of the sort key.

Special Case 1: Sorting a List of Dictionaries by the Value of a Field

```
list .sort (key= ... )
```

```
key= itemgetter('field')
```

- Let's consider a list of dictionaries
- In the simplest cases, the sort key corresponds to one of the dictionary fields .
- Examples:
  - Sort by last name
  - Sort by ID number
- For these cases, Python provides the `operator.itemgetter ()` function that implements a sort key to select the field of interest, specifying the field name as an argument:
  - `key= operator.itemgetter('field_name')`

Example: Sorting a list of dictionaries

```
from operator import itemgetter

# reading order from file
print(students[0], students[1], students[2])

# sort by ID number
students.sort(key=itemgetter('ID'))
print(students[0], students[1], students[2])

# sort by name
students.sort(key=itemgetter('name'))
print(students[0], students[1], students[2])

# sort by surname
students.sort(key=itemgetter('surname'))
print(students[0], students[1], students[2])
```

Special Case 2: Sorting a List of Lists by the Value of a Field

```
list .sort (key= ... )
```

```
key= itemgetter (index)
```

- Let's consider a list of lists (table)
  - The same reasoning applies to a list of tuples.
- In the simplest case, the sort key corresponds to one of the columns of the table
- Examples:
  - Sort by the value of the first column
  - Sort by the value in the fifth column
- The `operator.itemgetter ()` function can be used also in this case, specifying the integer (index) of the column to sort on:
  - `key= operator.itemgetter(num_column)`

Example: Sorting a list of lists

- Using itemgetter is possible even for lists of lists, or lists of tuples
- In this case the argument of `itemgetter` is **the index** of the column to use as sort key

```
[['A', '2'],
 ['A', '4'],
 ['A', '3'],
 ['C', '4'],
 ['D', '4'],
 ['C', '1'],
 ['B', '4'],
 ['D', '1'],
 ['D', '3'],
 ['B', '1'],
 ['C', '2'],
 ['A', '1'],
 ['C', '3'],
 ['B', '2'],
 ['B', '3'],
 ['D', '2']]
```

```
table.sort(key= itemgetter(0))
# sort by first element
```

```
[['A', '1'],
 ['A', '4'],
 ['A', '2'],
 ['A', '3'],
 ['B', '2'],
 ['B', '4'],
 ['B', '3'],
 ['B', '1'],
 ['C', '2'],
 ['C', '4'],
 ['C', '3'],
 ['C', '1'],
 ['D', '1'],
 ['D', '2'],
 ['D', '4'],
 ['D', '3']]
```

```
table.sort(key=itemgetter(1))
# sort by second element
```

```
[['D', '1'],
 ['C', '1'],
 ['A', '1'],
 ['B', '1'],
 ['D', '2'],
 ['B', '2'],
 ['C', '2'],
 ['A', '2'],
 ['B', '3'],
 ['D', '3'],
 ['A', '3'],
 ['C', '3'],
 ['C', '4'],
 ['A', '4'],
 ['B', '4'],
 ['D', '4']]
```

Special Case 3: Using **Multiple** Cascading Sorting Criteria

```
list .sort (key= ... )
```

```
key= itemgetter ( a,b,c )
```

Advanced topic

- Sometimes, a single field (or a single column) is not enough to specify the complete sort order.

- Examples:
  - Sort students by surname, and if multiple students have the same surname, sort them by first name
  - Sort the coordinates by abscissa, and if the abscissa is the same, by ordinate.

- You can exploit the fact that itemgetter can receive multiple parameters, which correspond to the keys (for dict) or indices (for list) to sort by.
  - `list.sort (key=operator.itemgetter ('surname', 'name'))`
  - `coordinate.sort (key= operator.itemgetter (0, 1))`

Special Case 4: Using Functions

```
list .sort (key= ... )
```

```
key= len
```

Advanced topic

- You can pass to the key= parameter any built-in Python function (or even a function that you implemented) as long as it returns an output that is comparable (e.g. a number)
  - sum, min, max, len , abs , round… (see unit P5)
- Examples:
  - Sort a list of sets, based on the increasing number of elements → the key function is `len ()`
  - Sort a list of lists (table), containing numbers, based on the sum of the values of each row → the key function is `sum()`
- Solutions:
  - `sets.sort (key= len )`
  - `table.sort (key= sum )`
- ⚠ Attention! ⚠
  You must indicate the name of the function as the key, without the round parentheses . In fact, we do not have to invoke the function, the sort algorithm will invoke it.

Complex Data Structures

8.3

Complex Structures

- Containers are very useful for storing collections of values
  - In Python, the list and dictionary containers can contain any type of data, including other containers
- Some data collections, however, may require more complex structures
  - In this section, we explore problems that require the use of a complex structure

Example: A Dictionary of Sets

- The index of a book specifies on which pages each term occurs
- Build a book index from page numbers and terms contained in a text file with the following format:

```
6:type
7:example
7:index
7:program
8:type
10:example
11:program
20:set
```

Note that the same term may appear multiple times, in different pages

Example: A Dictionary of Sets

- The file includes every occurrence of every term to be included in the index and the page on which the term appears

- If a term appears on the same page more than once, the index **should include the page number only once**

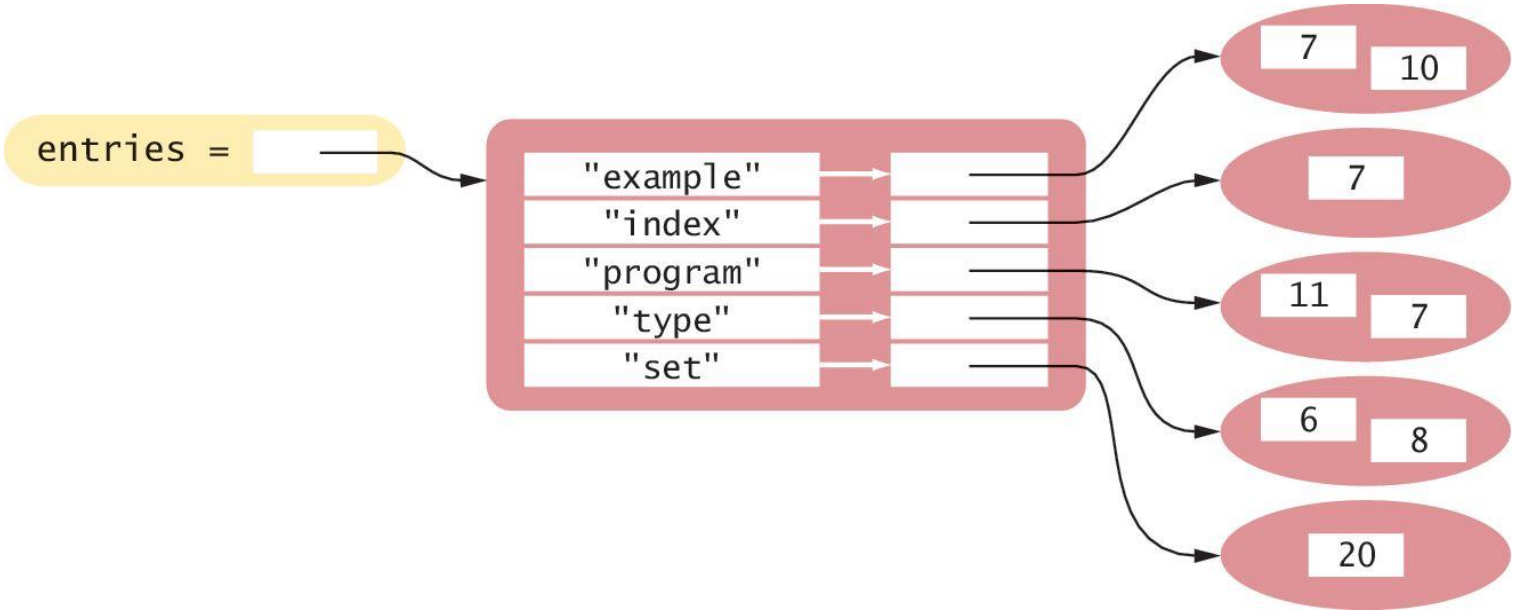
Example: A Dictionary of Sets

- The output of the program should be a list of terms in alphabetical order followed by the page numbers on which the term appears, separated by commas, like this:

```
example: 7, 10
index: 7
program: 7, 11
type: 6, 8
set: 20
```

Example: A Dictionary of Sets

- A dictionary of sets would be appropriate for this problem
- Each term becomes a **key**, and its corresponding **value** is a set of the page numbers where it occurs



Why Use a Dictionary?

- The terms in the index must be unique
  - By making each term a dictionary key, there will be only one instance of each term.
- The index listing must be provided in alphabetical order by term
  - We can iterate over the keys of the dictionary in sorted order to produce the listing
- Duplicate page numbers for a term should only be included once
  - By adding each page number to a set, we ensure that no duplicates will be added
  - We don’t need another dictionary because there is no extra information associated to page numbers

Dictionary Sets: `Buildindex.py`

```
 5  def main() :
 6      # Create an empty dictionary.
 7      indexEntries = {}
 8
 9      # Extract the data from the text file.
10      infile = open("indexdata.txt", "r")
11      fields = extractRecord(infile)
12      while len(fields) > 0 :
13          addWord(indexEntries, fields[1], fields[0])
14          fields = extractRecord(infile)
15
16      infile.close()
17
18      # Print the index listing.
19      printIndex(indexEntries)
```

Dictionary Sets: `Buildindex.py`

```
26  def extractRecord(infile) :
27      line = infile.readline()
28      if line != "" :
29          fields = line.split(":")
30          page = int(fields[0])
31          term = fields[1].rstrip()
32          return [page, term]
33      else :
34          return []
```

Dictionary Sets: `Buildindex.py`

```
41  def addWord(entries, term, page) :
42      # If the term is already in the dictionary, add the page to the set.
43      if term in entries :
44          pageSet = entries[term]
45          pageSet.add(page)
46
47      # Otherwise, create a new set that contains the page and add an entry.
48      else :
49          pageSet = set([page])
50          entries[term] = pageSet
```

Dictionary Sets: `Buildindex.py`

```
56      for key in sorted(entries) :
57          print(key, end=" ")
58          pageSet = entries[key]
59          first = True
60          for page in sorted(pageSet) :
61              if first :
62                  print(page, end="")
63                  first = False
64              else :
65                  print(",", page, end="")
66
67          print()
```

Example: A Dictionary of Lists

- A common use of dictionaries in Python is to store a collection of lists in which each list is associated with a unique name or key
- For example, consider the problem of extracting data from a text file that represents the yearly sales of different ice cream flavors in multiple stores of a retail ice cream company

```
vanilla:8580.0:7201.25:8900.0
chocolate:10225.25:9025.0:9505.0
rocky road:6700.1:5012.45:6011.0
strawberry:9285.15:8276.1:8705.0
cookie dough:7901.25:4267.0:7056.5
```

Example: A Dictionary of Lists

- A comm… of lists in …
- For exa… file tha… multip…

Flavour	Store 1	Store 2	Store 3
Vanilla	8580.0	7201.25	8900.0
Chocolate	10225.25	9025.0	9505.0
Rocky road	6700.1	5012.45	6011.0
Strawberry	9285.15	8276.1	8705.0
Cookie Dough	7901.25	4267.0	7056.5

```
vanilla:8580.0:7201.25:8900.0
chocolate:10225.25:9025.0:9505.0
rocky road:6700.1:5012.45:6011.0
strawberry:9285.15:8276.1:8705.0
cookie dough:7901.25:4267.0:7056.5
```

Example: A Dictionary of Lists

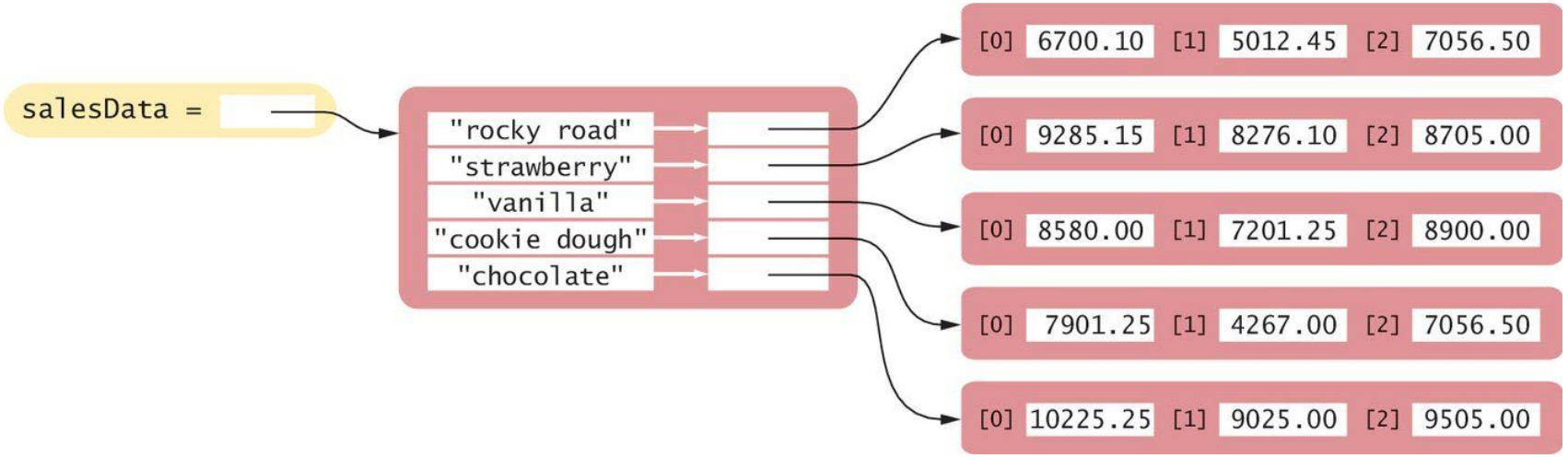
- The data should be processed to produce a report similar to the following:

				Total per flavour
chocolate	10225.25	9025.00	9505.00	28755.25
cookie dough	7901.25	4267.00	7056.50	19224.75
rocky road	6700.10	5012.45	6011.00	17723.55
strawberry	9285.15	8276.10	8705.00	26266.25
vanilla	8580.00	7201.25	8900.00	24681.25
Total per store	42691.75	33781.80	40177.50	

- A simple list is not the best choice:
  - The entries consist of strings and floating-point values, and they have to be **sorted by the flavor name**

Example: A Dictionary of Lists

- We can use instead a **dictionary**, with **one element per flavor**.
  - With this structure, each row of the table is an item in the dictionary
  - The name of the ice cream flavor is the key used to identify a particular row in the table.
  - The value for each key is a list that contains the sales, store by store, for that flavor of ice cream



Example: A Dictionary of Lists

- Let’s try to solve it together…

Summary

Python Sets

- A `set` stores a collection of unique values
- A set is created using a set literal `{…}` or the `set()` function
- The `in` operator is used to test whether an element is a member of a set
- New elements can be added using the `add()` method
- Use the `discard()` method to remove elements from a set
- The `issubset()` method tests whether one set is a subset of another set

Python Sets

- The `union()` method produces a new set that contains the elements in both sets
- The `intersection()` method produces a new set with the elements that are contained in both sets
- The `difference()` method produces a new set with the elements that belong to the first set but not the second
- The implementation of sets arrange the elements in the set so that they can be located quickly

Python Dictionaries

- A dictionary keeps associations between keys and values
- Use the `[]` operator to access the value associated with a key
  - Use the `get()` method to define a default value if the key isn't found
- The `in` operator is used to test whether a key is in a dictionary
- New entries can be added, or existing entries can be modified using the `[]` operator
- Use the `pop()` method to remove a dictionary entry